

Lab 1: Dart Fundamentals — Build a Course Roster Console App

Sep 21, 2026 · @Someone

A single 60–90 minute console-app lab that exercises every concept from Session 1 (Dart Syntax, Types & Control Flow) in one cohesive build — no Flutter yet, pure Dart in DartPad.

Overview

Students work individually in DartPad and build one cohesive console program — a **Course Roster Console App** — in a single sitting.

Format: Individual work, DartPad (dartpad.dev) or local Dart SDK. **Duration:** 60 minutes required (Parts 1–7) + up to 30 minutes optional stretch (Part 8). **Deliverable:** A link to your GitHub repository containing `course_roster.dart`.

Learning objectives

Objective	Lecture Topic (Session 1)
Write a correctly structured <code>main()</code> and helper function	Hello Dart · Anatomy of a Dart Function
Choose correctly among <code>var</code> , <code>final</code> , <code>const</code> , and explicit types	Dart's Type System · <code>var</code> / <code>final</code> / <code>const</code> & static
Model data with <code>String</code> , <code>int</code> , <code>double</code> , <code>bool</code> , <code>List</code> , <code>Map</code> , <code>Set</code>	Core Types at a Glance
Apply sound null safety with <code>?</code> , <code>??</code> , <code>??=</code> , <code>?..</code> , <code>!</code> , <code>late</code>	Sound Null Safety Essentials · Null-Aware Operators
Use <code>~/</code> , <code>%</code> , <code>is</code> / <code>is!</code> , and <code>cascade</code> (<code>..</code> / <code>?..</code>) correctly	Dart Operators · the two "Operators, Simplified" slides
Control flow with <code>if/else</code> , <code>switch</code> , and the ternary operator	<code>if</code> / <code>else</code> & <code>switch</code> · Ternary Operator & Fall-Through
Choose the right loop — <code>for</code> , <code>for-in</code> , <code>forEach</code> , <code>while</code> , <code>do-while</code>	Loops in Dart
Build a collection using <code>if/for</code> embedded in a literal	Control Flow Inside Collection Literals

Setup, Time Budget & Submission

Setup: Open dartpad.dev — no install needed. Prefer a local editor? Install the Dart SDK and run files with `dart run course_roster.dart`.

Working style: Build the file top to bottom, part by part. Run and check the console output after each part before moving on — small, frequent checks beat one big attempt at the end.

Time budget

Part	Minutes	Concepts
1 — Setup & Welcome	5	<code>main()</code> , <code>print</code> , doc comments
2 — Course & Roster Data	15	Types, <code>var/final/const</code> , <code>List/Map/Set</code>
3 — Null-Safe Instructor Info	10	<code>?</code> , <code>??</code> , <code>!</code> , <code>late</code>
4 — Formatting Strings	10	Interpolation, string methods
5 — Operators in Action	10	<code>~/</code> , <code>%</code> , <code>is/is!</code> , <code>..</code> , <code>?..</code> , <code>??=</code>
6 — Enrollment Logic	10	<code>if/else</code> , <code>switch</code> , ternary
7 — Reports & Loops	10	<code>for-in</code> , <code>forEach</code> , collection literals
Required subtotal	60	
8 — Stretch Goals (optional)	up to 30	CLI args, <code>dart format</code> , <code>dart analyze</code>
Total	60–90	

Submission

1. Create a **public GitHub repository** named `course-roster-lab` (or similar).
2. Add your completed `course_roster.dart` file, plus a short `README.md` with your name and a note on which parts (including any Part 8 stretch goals) you completed.
3. Commit and push your work.
4. Submit the **link to your GitHub repository** — via WhatsApp to your CR.

Core Tasks — Parts 1–4

Part 1 — Setup & Welcome (5 min)

Write `main()` and one helper function that follows the `returnType name(params) { body }` pattern from lecture.

1. Write a function `void printWelcome(String appName)` that prints a banner, e.g. `=== $appName ===`.
2. Give it a `///` doc comment describing what it does.
3. Call it from `main()` with the app name `'Course Roster Manager'`.

Expected output:

```
=== Course Roster Manager ===
```

Part 2 — Course & Roster Data (15 min)

Inside `main()`, declare the following, choosing `var`, `final`, or `const` deliberately for each (not all `var`):

- `const int maxCapacity = 4;` — a true compile-time constant
- `final DateTime createdAt = DateTime.now();` — known only at runtime, set once
- `String courseTitle = 'CS201: Mobile App Development';`
- `int capacity = maxCapacity;`
- `double creditHours = 3.0;`
- `bool isOpen = true;`
- `List<String> enrolledStudents = ['Aiden', 'Maria', 'Jamal'];`
- `Set<String> waitlist = {'Priya', 'Noah'};`
- `Map<String, int> attendanceCount = {'Aiden': 3, 'Maria': 4, 'Jamal': 2};`

Print `courseTitle`, `capacity`, and `enrolledStudents.length` in one interpolated string.

Check yourself: which of the nine declarations above legally could NOT be `const`? Why not? (Hint: which ones aren't known until the program runs?)

Part 3 — Null-Safe Instructor Info (10 min)

1. Declare `String? instructorEmail;` (leave it unassigned — it defaults to `null`).
2. Print it safely with a fallback: `print(instructorEmail ?? 'TBA');`
3. Declare `late String enrollmentCode;` and assign it inside a small setup function you write, e.g. `String generateCode(String title) => title.substring(0, 2).toUpperCase() + '101';` — call it and assign the result to `enrollmentCode` before printing it.
4. **Deliberate crash, then fix:** temporarily write `print(instructorEmail!.length);` and run it. Read the runtime error. Then replace it with a safe version using `??` or `?.` and confirm it no longer crashes. Leave the safe version in your final file.

Part 4 — Formatting Strings (10 min)

1. Given `String rawNames = ' Aiden , maria ,JAMAL , Priya '`, use `.split(',')` and a `for-in` loop with `.trim()` to build a new `List<String> cleanNames` with tidy capitalization-free values (don't worry about fixing capitalization — just trim whitespace).
2. Build one multi-line course description using a triple-quoted string (`'''...'''`).
3. Print a summary line using `${}` interpolation that embeds an expression, e.g. `'Seats left: ${capacity - enrolledStudents.length}'`.

Core Tasks — Parts 5–7

Part 5 — Operators in Action (10 min)

1. `~/ and %:` the room needs to be split into lab groups of 3. Compute `int fullGroups = enrolledStudents.length ~/ 3`; and `int leftover = enrolledStudents.length % 3`. Print both.
2. `is / is!:` declare `Object formInput = 'twenty-two'`; (simulating a value read from an untyped source). Write an `if (formInput is String)` branch and an `if (formInput is! int)` branch, each printing a message.
3. **Cascade ...:** build a report using `StringBuffer()` and three chained `.write(...)` calls (title, capacity, roster count), then print `.toString()`.
4. **Null-safe cascade ?..:** declare `List<String>? extraNotes`; (leave it null) and attempt `extraNotes?.add('Room change pending')`. Print `extraNotes` afterward and confirm it's still null — nothing ran.
5. `??=:` declare `int? bonusSeats`; then `bonusSeats ??= 0`; Print it.

Part 6 — Enrollment Logic (10 min)

1. **if/else:** write a check — can a new student enroll? `if (isOpen && enrolledStudents.length < capacity) { ... } else { ... }` — print an appropriate message either way.
2. **switch:** declare `int enrollmentStatusCode = 200`; and write a `switch` with cases `200 → 'Enrolled'`, `404 → 'Course not found'`, and a default `→ 'Unknown error'`. **Every case needs a break.**
3. **Ternary:** build a one-line tag: `String statusTag = isOpen ? 'OPEN' : 'FULL'`; and print it.

Part 7 — Reports & Loops (10 min)

1. **for-in:** print the full roster, one name per line.
2. **forEach:** print every entry in `attendanceCount` using the two-parameter (`key, value`) callback.
3. **Collection literal with embedded if/for:** build

```
List<String> announcements = [  
    'Welcome to $courseTitle',  
    if (!isOpen) 'Course is FULL – waitlist open',  
    for (var student in waitlist) 'Reminder: $student, please confirm attendance',  
];
```

then print each announcement with a `for-in` loop.

Part 8 — Stretch Goals (optional, up to 30 min)

For students who finish Parts 1–7 early. None of this is required for full credit.

1. **CLI arguments:** change `main()` to `void main(List<String> args)`. If `args` is not empty, use `args[0]` as the course title instead of the hardcoded one. Test with `dart run course_roster.dart "CS210: Databases"`.
2. **Tooling:** run `dart format .` and `dart analyze` on your file. Fix any warnings. Paste the final analyzer output (ideally `No issues found!`) into your submission.
3. **Beyond the lecture (preview of Session 2):** if you're curious, look up `Set.union()` in the Dart docs and try merging waitlist with a second course's waitlist. This uses a method not yet covered in lecture — a nice preview, not a requirement.

Sample final console output

Your exact wording may differ — this shows the *shape* of a complete run:

```
=== Course Roster Manager ===
CS201: Mobile App Development | Capacity: 4 | Enrolled: 3
TBA
Enrollment code: CS101
Full groups of 3: 1, leftover: 0
This is text!
Report: CS201: Mobile App Development | Cap: 4 | Roster: 3
Extra notes: null
Bonus seats: 0
You're in! Welcome aboard.
Enrolled
OPEN
Aiden
Maria
Jamal
Aiden: 3
Maria: 4
Jamal: 2
Welcome to CS201: Mobile App Development
Reminder: Priya, please confirm attendance
Reminder: Noah, please confirm attendance
```

Starter skeleton

Paste this into DartPad and fill in each `// TODO`:

```
/// Course Roster Console App – Lab 1
/// Fill in each TODO. Run after every part to check your output.

void main() {
  // ----- Part 1: Setup & Welcome -----
  // TODO: call printWelcome('Course Roster Manager');

  // ----- Part 2: Course & Roster Data -----
  // TODO: const maxCapacity, final createdAt, courseTitle, capacity,
  //       creditHours, isOpen, enrolledStudents, waitlist, attendanceCount
```

```
// ----- Part 3: Null-Safe Instructor Info -----
// TODO: instructorEmail, late enrollmentCode, safe printing

// ----- Part 4: Formatting Strings -----
// TODO: rawNames -> cleanNames, multi-line description, interpolation

// ----- Part 5: Operators in Action -----
// TODO: ~/ and %, is / is!, cascade .., ?.., ??=

// ----- Part 6: Enrollment Logic -----
// TODO: if/else, switch (with break!), ternary

// ----- Part 7: Reports & Loops -----
// TODO: for-in, forEach, collection literal with if/for

// ----- Part 8: Stretch (optional) -----
}

/// TODO Part 1: doc comment + printWelcome(String appName)
```

Self-Check Checklist

Before submitting, confirm each of these is true of your file:

- ☐ dart analyze reports no errors (warnings are OK but worth reading)
- ☐ Used at least one `const`, one `final`, and one plain `var`/typed declaration — each chosen deliberately
- ☐ Used `List<String>`, `Map<String, int>`, and `Set<String>` each at least once
- ☐ Handled a nullable value with `??` or `?..` — no `!` left on a value that could realistically be null
- ☐ Used `late` for a field set after declaration, not at declaration
- ☐ Used `~/` and `%` together correctly
- ☐ Used `is` or `is!` at least once
- ☐ Used `cascade ..` at least once, and `?..` on a nullable object
- ☐ Used `??=` at least once
- ☐ Both `if/else` and a `switch` (every case ending in `break`) appear
- ☐ Used the ternary operator at least once
- ☐ Both `for-in` and `forEach` appear
- ☐ At least one collection literal uses an embedded `if` and/or `for`
- ☐ Program compiles and runs, producing console output resembling the sample

Instructor: suggested weighting — 70% checklist completion, 20% runs without errors, 10% code clarity/comments. Adjust to your rubric.

Common Pitfalls (from lecture — watch for these)

Pitfall	Fix
Forgetting the semicolon	Dart never infers statement ends — every

Pitfall	Fix
	statement needs ;
const where final was needed	If the analyzer says "not a constant expression," switch const → final
Missing break in a switch	Causes silent fall-through into the next case
Overusing !	Each ! is a promise to the compiler — prefer ?? or a null check when there's real doubt
if (someInt)	Dart has no truthiness — write a real comparison, e.g. if (someInt != 0)